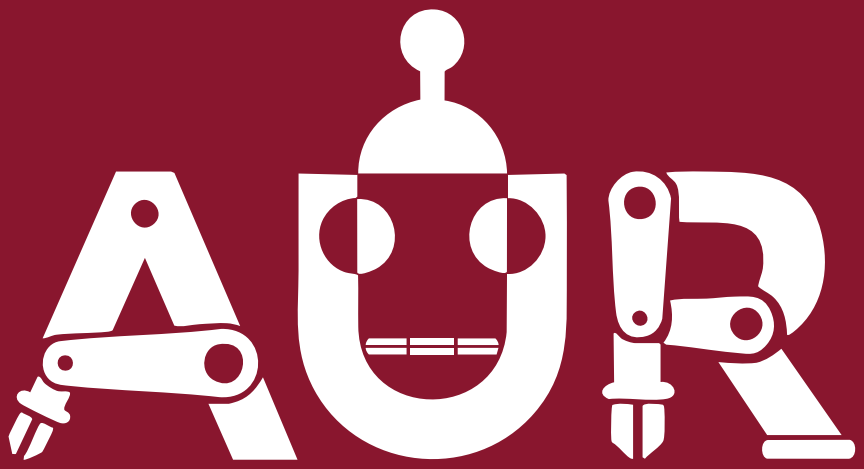

Training '26

Software | Phase II

Task 3: Advanced Python

Published: Thursday 27th, August 2026

Introduction

After watching Session 3, you should be able to use python with OOP and in a cleaner, more advanced manner.

> **Work in github**
>
> your task must be uploaded in github as well in a certain format: read the submission guidelines. Make sure that ALL your subtasks are created under a folder called task_2, each subtask's files inside a folder called subtask_i where i is the subtask's number

Task — A Small Library Management System

Design and implement a simplified library system that models books, members, and loans.

1 Requirements

1. **Polymorphism.** Create an abstract base class `LibraryItem` with subclasses `Book`, `DVD`, and `Magazine`. Each subclass defines its own loan period (Book = 21 days, DVD = 5 days, Magazine = 14 days).
2. **Encapsulation.** Each item tracks its own status internally. This state must not be directly settable from outside the class (no `item.status = ...` from calling code). Instead, expose behaviour through methods such as `checkout()`, `return_item()`, and `mark_lost()` that perform the transition and any validation (e.g. you can't check out an item that's already checked out).
3. **Enum, not magic strings.** Create an `ItemStatus` enum with members `AVAILABLE`, `CHECKED_OUT`, and `LOST`. All status comparisons and assignments should go through this enum.
4. **Comparable & printable.** Items should sort alphabetically by title — implement `__lt__` so `sorted(items)` works with no `key=`. Implement `__repr__` (unambiguous, debug-friendly) and `__str__` (human-readable, e.g. `"Dune (Book) — Available"`).
5. **Alternative constructor.** Add a `@classmethod` called `from_dict` that builds the correct concrete item from a dictionary parsed from a line of the `database.txt` file (see format below).
6. **Static method.** Add a `@staticmethod` that validates an ISBN checksum (ISBN-10 or ISBN-13 - your choice, but document which one). It should take an ISBN string and return `True`/`False`, using no `self` or `cls`.
7. **Single Responsibility Principle.** Create a `Library` class that manages the collection and check-outs (`add_item`, `checkout`, `return_item`, `find_by_title`, `list_available`, etc.) — it must **not** read or write files itself. Create a separate `Database` class responsible only for saving/loading the collection to/from `database.txt`. `Library` may **use** a `Database` instance, but persistence logic must live only in `Database`.
8. **Open/Closed Principle.** Design it so adding a new item type (e.g. `AudioBook`) requires writing a new subclass only — no edits to `Library`'s core logic. (Hint: `Library` should operate on `LibraryItem` polymorphically, and `from_dict` dispatch should be driven by a type-registry or lookup dict, not an `if/elif` chain checked into `Library`.)

Bonus: Make `Database` a singleton, so every part of the program shares one instance and one open connection to the file.

database.txt example

One item per line, pipe-delimited:

```
type=Book|title=Dune|author=Frank Herbert|isbn=9780441013593|status=AVAILABLE
type=DVD|title=Inception|director=Christopher Nolan|status=CHECKED_OUT type=Magazine|title=National Geographic|issue=2026-08|status=AVAILABLE
```

Fields vary by type - `from_dict` should only require the fields relevant to that subclass.
Notice the text is written in a form that when retrieved and parsed in a list of dictionaries variable, you can easily call `from_dict()` for each dictionary element

Submission

Github Steps just like the last task sequence of steps, you should be able to conclude it on your own, but let's revise the steps once again:

1. merge the task-2 pull request then delete the task-2 branch since no more development on this branch will be done - stale branches should be deleted to retain a clean working environment
2. create a branch called task-3
3. create a folder called task_3
4. work on your project, commit after finishing some relevant work - make sure to make a good commit title
5. push your work on the branch task-3
6. cerate a pull request to merge task-3 into main - **DO NOT merge the pull request**

Besides Github you must upload the task_3 folder as a zip file in the follwing link:

https://forms.gle/pwvzmK4VWaqd4FWV7

Deadline: Saturday, August 29th -- 11:59 pm